

Esercizio 1. Programmazione concorrente

Si scriva un programma in cui ogni thread rappresenta un utente che, iterativamente

- 1) prendere una certa risorsa dal deposito delle risorse;
- 2) usa la risorsa;
- 3) rilascia la risorsa.

Se la risorsa che un utente A cerca di prendere al passo 1) e' occupata perche l'ha presa un altro utente B, occorre che A attenda che B rilasci la risorsa.

Si modifichi opportunamente il codice dato, nel quale manca qualunque sincronizzazione tra i thread. Bisogna che se un utente viene messo in attesa del rilascio di una risorsa, venga poi svegliato non appena la risorsa torna disponibile.

NB: la natura delle risorse e l'uso che se ne fa sono irrilevanti.

Occorre evitare i problemi tipici della programmazione concorrente (corse critiche, deadlock, starvation, ecc.).

Si ricorda che bisogna caricare i file .java, NON i .class, raggruppati in un unico file zip.

Esercizio 2. Programmazione distribuita / socket

Si chiede di trasformare il programma sviluppato come soluzione dell'esercizio di programmazione concorrente in un sistema distribuito, in cui il server gestisce il deposito delle risorse e i client operano come utenti.

Si realizzi il sistema mediante l'uso di socket.

Come sempre, occorre evitare i problemi tipici della programmazione concorrente (corse critiche, deadlock, starvation, ecc.).

Si ricorda che bisogna caricare i file .java, NON i .class, raggruppati in un unico file zip.

Esercizio 3. Programmazione distribuita / RMI

Si chiede di trasformare il programma sviluppato come soluzione dell'esercizio di programmazione concorrente in un sistema distribuito, in cui il server gestisce il deposito delle risorse e i client operano come utenti.

Si realizzi il sistema mediante l'uso di RMI.

Come sempre, occorre evitare i problemi tipici della programmazione concorrente (corse critiche, deadlock, starvation, ecc.).

Si ricorda che bisogna caricare i file .java, NON i .class, raggruppati in un unico file zip.